LEUPHANA
UNIVERSITÄT LÜNEBURG

Field of study: Master of Science Management and Data Science

Project on the module Optimization Techniques

Implementation And Visualization Of Lost Target Search Using Motion-Encoded PSO

Submitted by
Sthitadhee Panthadas (3044093)

Lüneburg, 14.10.2022

Examiner: Prof. Dr. Ing. Paolo Mercorelli

Statuatory declaration

I hereby certify that this project has been written independently and that no sources or aids other than those indicated have been used, and that all passages in the thesis that have been taken verbatim or in spirit from other sources have been marked as such.

Lüneburg, 14.10.2022

Sthitadhee Panthadas

Abstract The project implements a paper that solves an optimization problem on lost target search using UAVs which still is a highly computationally costly and complex problem. Lost target search is also a well researched topic as a result this forms a great candidate for deeper understanding. So, the project tries to mirror the results in an environment of a browser with an effort to better break down the steps of the algorithm. The implications of the project are very simple yet powerful. The aim of the project is to give a more explicitly clear picture of the programmatic actions of the motion-encoded PSO in lost search operations. The breakdown is also quite useful for academics coming to study such optimization problems.

The project reinvents the implementation using JavaScript, Python, HTML and CSS from Matlab code. The backend or server was implemented in Python using Flask. While VScode was the go-to text editor for the making and running of the code during the development process.

In the results, it can be seen the work is able to show clear steps of the algorithms and also allows visual aids for any user of the web page. The project also illustrates many sides that might not be immediately realized from the paper's implementation. Overall, the project can be an effective as a tool for experimentation, learning and even research.

Contents

1 Introduction

The project work builds on the work and implementation of Phung and Ha, 2020 which delineates the theoretical and mathematical design and build up of target or cost function for a lost target search. Motion Encoded PSO later optimizes this target function to obtain the optimized search path for Unmanned aerial vehicle (UAV) representing the highest probability of finding the target. The project handles a specific portion of this paper and implements the visualization of the search area, UAV flight path and target detection from start to end highlighting the steps of the Motion-encoded particle swarm optimization (MPSO).

In research, many papers determine the cost function based on a Bayesian approach that calculates the probability of each UAV flight path. This is often the case Bourgault et al., 2003b, Bourgault et al., 2003a, Lanillos et al., 2013, Wong et al., 2005 because many search problems are defined by probabilistic formulations. In lost target search, most often there is an opportune critical period where the probability of locating the target is maximum. This paper takes advantage of that concept.

Much of the literature of Phung and Ha, 2020 involves 3 parts - different search environments, target dynamics and sensor models. These three components adequately describe the entire searching problem having the complexity level from being a non-deterministic polynomial time hardness or NP hard problem to non deterministic exponential time completeness (NEXP-complete). The paper choses MPSO as their optimization algorithm. because it incorporates the fast convergence and simplicity of particle swarm optimization (PSO) and also allows for proper representation of nodes or UAV positions in motion space. But in addition, there are many other optimization techniques such as Gradient descent (GD), Ant colony optimization (ACO), Cross entropy optimization (CEO) mentioned in the introduction of the paper Phung and Ha, 2020 used for other researches in this area of LTS. Due to the many uncertainties and conditional and complex form of the problem, it is important to note that search problems having altering and complex location dynamics, harsh environmental conditions, fast moving or eluding movements are still very challenging to solve which

makes such problems more compelling to investigate.

Being motivated from the fascination of solving search based problems and the advantages of PSO, the project has the following contributions. The contributions include: 1. Complete visualization of the MPSO process, 2. Pinpoint the chief parameters of the whole target search operation from the settings section of the application, 3. A Mini Map of the process that indicates visually the current global best for the ease of following the process, 4. A simple info panel that delineates the current ongoing process of the running optimization algorithm of the lost target search optimization project.

The rest of the structure of the report is the following. `Methodology` will be a detailing of the grounding knowledge or `theoretical design` of the whole process implemented in the paper Phung and Ha, 2020 which includes all the necessary theoretical concepts and mathematical formulation in a concise format. It will also contain the explanation for the `Project Design` to develop the web application for the demonstration of the whole MPSO process. In `Results` we will showcase the outcome of the project work. `Discussions` presents the discussion including some limitations and `Reflection on the Project` will represent the overall work plan. There will also be supplementary materials, list of figures and list of acronyms.

2 Methodology

2.1 Theoretical Design

The optimization problem for lost target search requires the following three modeling concepts to build the cost function to maximize - target model, sensor model and the belief map updates. The following is a compressed understanding of the paper Phung and Ha, 2020.

Target The target is defined by the notation $x \in X$ where x represents all the possible locations of the target from first known location to the end of the search process.

Belief Map The belief map is a discretized searching space for the UAV to search for the target. S is the searching space where S_r is the number of rows of the map and S_c is the number of columns. Probabilities for each cell of the belief map is predetermined based on the target position, UAV position and search conditions. The target mean position and variance is determined during the first determination of location of the target position. Using this mean and variance a PDF distribution is used to create the probability map for the target represented by b(x), the belief map. Probability of the target being in the belief map always sums to 1.

Sensor A detection algorithm is used here to make independent observations z_t at time step t. The detection algorithm has two possible events - detection event, D_t, or non detection event, $\bar{D}_t$. Every observation is independent of each other.

Target Dynamics Target movement here is modeled using the Markov process and the probabilities from x_{t-1} to x_t is known for cells of the belief map represented by the transition function $p(x_t \mid x_{t-1})$. The target is assumed to be always present within the belief map and it is presumed that the target path is completely known.

Sensor Model Knowing the UAV sensors are not always accurate and have varying uncertainties. Thus, the observation likelihood is mathematically represented by $p(z_t \mid x_t)$ given there is certain existing knowledge of the sensor model.

Belief Map Update The belief map is initialized from the first detection point of the target. The target position is defined by the UAV in the map with a mean and a variance of target at the first detection point. The belief map is then updated using a Bayesian approach where there are two parts - a prediction and an observation. The predicted belief map probabilities are calculated by taking into consideration all the possible moves of the target. The **predicted** belief map calculation has thus the formulae given in equation 2.1.

$$\hat{b}(x_t) = \sum_{x_{t-1} \in S} p(x_t \mid x_{t-1}) b(x_{t-1}) \tag{2.1}$$

The observation probabilities will be available based on the detection or no detection event. We can have the updated belief map just by multiplying the predicted belief map with the new conditional observation likelihood. It can be noticed that equation 2.2 takes advantage of the Bayesian Theory.

$$b(x_t) = \eta_t p(z_t \mid x_t) \hat{b}(x_t) \tag{2.2}$$

Where,

$$1/\eta_t = \sum_{x_t \in S} p(z_t \mid x_t) \hat{b}(x_t) \tag{2.3}$$

Realization of the Searching Objective Function Looking at equation 2.2 where we can find the probability of the target not being detected at time t which is shown in equation 2.4. And the probability of the target being detected at time t would thus be equation 2.5.

$$r_t = p(\bar{D}_t \mid x_t) \hat{b}(x_t) \tag{2.4}$$

$$r_t = p(D_t \mid x_t) \hat{b}(x_t) \tag{2.5}$$

Then considering the joint probability of all the non detection from $t = 1$ to $t = t$ can be thus represented as equation 2.6 where R_t is joint probability of all the non detection events till time t while the probability of first detection at time t is p_t shown in equation 2.7. So, the cumulative probability of detecting at any point from the start time to time t represents the cost function shown in equation 2.8.

$$R_t = \prod_{k=1}^{t} r_t = R_{t-1} r_t \tag{2.6}$$

$$p_t = \prod_{k=1}^{t} r_t = R_{t-1}(1 - r_t) \tag{2.7}$$

$$P_t = \sum_{k=1}^{t} p_t = P_{t-1} + p_t = 1 - R_t \tag{2.8}$$

Motion Encoded Particle Swarm Optimization Particle Swarm Optimization is designed for solving optimization problems that have more than one local minima or local maxima. In motion encoded PSO, the cartesian space is converted to motion space. So, the velocity update equation of PSO is changed by encoding velocity to both direction and magnitude for the motion of the UAV. This gives a better implementation of the PSO for searching dynamically moving targets especially in multiplex dynamics.

$$v_{k+1} \leftarrow \omega v_k + \vartheta_1 r_1 (L_k - x_k) + \vartheta_2 r_2 (G_k - x_k)$$

$$x_{k+1} \leftarrow x_k + v_{k+1}$$

where ω is the inertial weight, φ_1 is cognitive coefficient, φ_2 is social coefficient, r_1 and r_2 uniform random sequence values for local and global best respectively, k is unit steps, x_k is position vector, v_k is velocity vector, L_k is local best and G_k is global best.

The pseudocode and the expanded version of the mathematics is provided in the paper Phung and Ha, 2020.

2.2 Project Design

The project is a new contribution in a sense that you can visualize the MPSO and thus have the feel for the optimization algorithm in action. You can also pinpoint the parameters of the project visually rather than going through code as the most salient parameters are indicated in the settings of the web application. This is why creation of this application would be useful especially for academics and students trying to picture the working of the target search operation performed by an UAV.

The 2 core building blocks of the project would be the HTML code along with the CSS code that builds up the design of the web page. The other building block would be the back-end python code that creates and spits out the first belief map and the JavaScript code in `simulation.js` file that incorporates the belief map within the design of the web pages that allows us to visualize the map.

```
PROJECT OPTIMIZATION TECHNIQUE/
|-- app/
|   |-- app.py
|
|-- web/
|   |-- static/
|   |   |-- css/
|   |   |   |-- bootstrap.min.css
|   |   |   |-- bootstrap.min.css.map
|   |   |   |-- style.css
|   |   |
|   |   |-- img/
|   |   |   |-- glyph-icons-settings-png
|   |   |
|   |   |-- js/
|   |   |   |-- helper.js
|   |   |   |-- map.js
|   |   |   |-- matlab.func.js
|   |   |   |-- mpso.js
|   |   |   |-- simulation.js
|   |   |   |-- variable.js
|   |   |
|   |
|   |-- templates/
|   |   |-- index.html
|   |
|
|-- venv
|-- README.md
|-- .gitignore
```

Figure 2.1 File Structure of the Project.

Figure 2.1 shows you the file structure of the project. The `app/` folder represents the folder for the server and `app.py` is responsible for the creation of the first belief map probabilities in a matrix format. Although originally it was thought that server would handle the belief map update, the work was later given to the front-end considering the time delay that might occur when fetching the data from the server and the creation of gigantic amounts of server requests from the front-end.

The `web/` folder secures the front-end of the project. The `templates/` folder inside the `web/` folder contains the HTML code for the web page that helps

draw everything. The `static/` folder withholds the CSS files for designing, image files for buttons (only one image is used so far), and JavaScript (JS) files for the requests and workings of the project. The JS files act as the engine of the whole project.

Diving deep into the `.js` files in the `js/` folder, `simulation.js` represents the index file or the primary file that starts the engine for the project. A point-wise explanation is given below.

simulaton.js

- Initializes the web page variables, model, target dynamics and button events
- Declares the button event for visualizing the mpso algorithm
- Creates the settings that helps manipulate the changes with the simulation
- Draws the initial belief map, Mini Map and Info panel

mpso.js

- Initializes all the MPSO parameters
- Draws the updates in the belief map, mini map and info panel
- Evaluates the cumulative probability and runs the MPSO
- Draws the final belief map

matlab.func.js

- Contains all the matlab code converted to JS

map.js

- Contains helper functions for drawing belief map, minimap and info panel

helper.js

- Contains helper functions for the overall project

variable.js

- Stores for all the global variables of the project

Overall, there are minor changes when compared to the Matlab code of the paper which is due to the difference of the languages used for implementation. The code logic remains the same with extra code for drawing each step and working of the MPSO algorithm.

All the functions are commented and explained in the code for better readability.

3 Results

3.1 Project Results

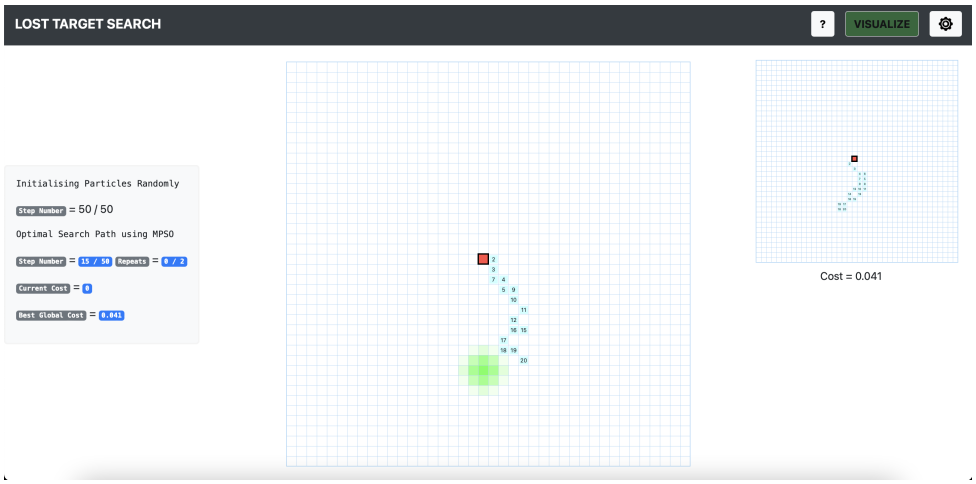

Figure 3.1 3 panels of the web page. Left side contains the info panel, middle has the belief map, and right side contains the mini map. The figure shows the algorithm in action.

There are 4 main components of the project. The most principal component is the belief map in the middle. Then there is an information box on the left. The belief map initially has the target position marked by square blocks of green which is the distribution of the target position according to the UAV sensor when the target was first detected.

Figure 3.2 Information Panel, Belief Map, Mini Map respectively.

The program runs when you click the `VISUALIZE` button. When one clicks the button another component, the Mini Map, will appear on the right. The above mentioned components are all separately shown in figure 3.2.

The Mini Map is responsible for showing the most updated global best path taken by the UAV and the most updated global cost of that path for the user's reference. The belief map is on the other hand responsible for showing the different random path generations of the UAV as well as visualizing the target movement determined by target's trajectory. On the info panel, the appropriate information can be found. Namely, the stage of the process (random initialization of path or mpso determined paths), the step number of the process, the number of repeats within the process, the current or local best (LB) and global best cost.

There are 2 other panels or dialog elements that are helpful for the project. The settings dialog and the Help or Question dialog. The settings dialog shown in figure 3.3 has most of the parameters with which you can change the simulation. For now, the options in the settings panel is a bit restricted due to not being able to test all the options but it allows us to know some of the parameters that eventually can be manually changed to change the working of the optimization process itself. The options available are Algorithm Type, Speed, Target Scenarios, UAV flight steps and other map and target dynamic parameters.

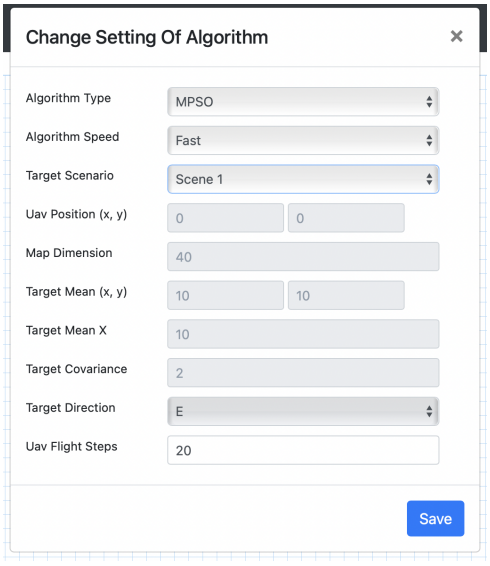

Figure 3.3 Settings Dialog.

You will notice Target Scenario is responsible for mutating the belief map and target dynamic parameters. While the Algorithm speed changes the speed of the simulation and UAV flight steps, which is an important constraint, changes the number of paths

taken by the UAV. For now the Algorithm Type is set to MPSO only but it will be possible to add and simulate any other optimization algorithm here if allowed for, which would be an interesting addition for the future.

In the Target Scenarios option you have 1 scene to play around with but the other five would be added later on in time.

Finally, in the help manual shown in figure 3.4, you will find all the details required to get started with the web application.

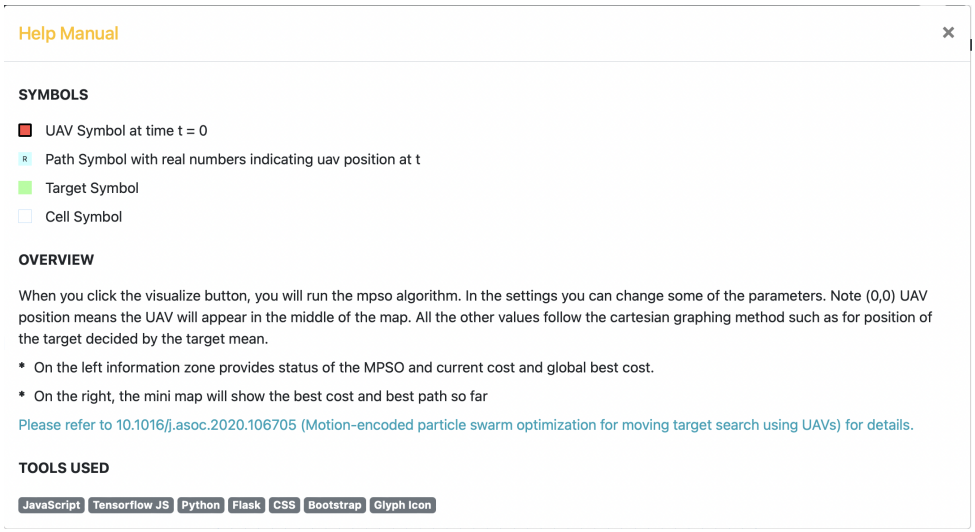

Figure 3.4 Help Dialog.

At the end of the process of the optimization algorithm you will see the results being shown on the info panel on the left and the belief map showing the final map result from the operation. The start of the process can be identified through the `VISUALIZE` button turning green and the end of the process can be understood when the button returns back to its initial form.

4 Discussion

Overall, the project tries to implement the paper visually in the form of a web page that handles user requests to implement the MPSO algorithm for lost target search. The project converts the Matlab code to partly Python and mostly JavaScript code so as to make the whole process more visual in the browsers. The project creates a web application with the main updating belief map in the center of the page, a minimap of the right corner to help the utilizers to capture the improvements caused by the MPSO algorithm and an info map to capture the related changing information based on the progress of the algorithm. Finally, there is a button for settings currently quite restricted that help you comprehend the factors that influence the visuals of the web page and the algorithm. You will also find a Help Manual for further information about the project.

Although it might be possible that due to the overflowing amount of visuals it might be somewhat difficult to understand the undergoing of the work for the first time. It is recommended to run the simulation on all the available options for speed for a customized feel of the ongoing process. It is also recommended to go through the paper once as well.

During the process of working with the project, some of the limitations of JS in providing accurate and expected mathematical results were obviously felt. The problems were deviated through creation of customized functions and tensorflow js. The idea was to keep the project as light weight and dependency free as possible.

As the paper states, in practical search processes, simple target dynamics as shown here may not be adequately sufficient. This is where techniques such as Kalman filtering may be utilized to provide more practically viable results. Also, in the implementation of the paper, all UAV paths consisting of duplicate repeating nodes were removed as results and only unique paths were considered. But for the project this has been ignored as they converge more quickly this way. Hence, one can see some of the missing digits in the path. Future work would consist of working with Kalman filter in lost target search as well as researching more on whether it would be also

useful to incorporate the UAV paths with repeating nodes as the UAV is searching around.

There are some limitations of the project as well. Due to limitations of time, not all the scenes of the paper have yet been implemented for the final project demonstration as they will require further extensive simulation testing. Thus, the settings are quite limited at the moment. Further, the code is not yet optimized fully and one can notice instances of repeating code. Finally, the web application project for lost target search is not optimized for smart phones, tabloids or any smaller screens than that of the Mac Air.

5 Reflection on Project Work

The work plan of the project was divided into Attending Classes, Research, Planing and Implemention, Presentation slides writing and Report writing. During Research, after reading about roughly 30 papers, this particular project felt to be the most interesting to work with. The plan of action and implementation of the paper sometimes intertwined as ongoing implementation led to some changes of the initial scheme.

For the coding process, VS code was used for the role of the text editor. The Matlab environment was used for research, comprehension and debugging of the author's code. Languages required for the whole project completion were Python, CSS, HTML, JavaScript and Matlab. For version control, git was used to organize everything. Finally, for fast and simple hosting Heroku, a cloud platform as a service (PaaS) that supports python deployment, was used to deploy the web application.

The program is tested on Safari and Chrome on Mac.

The report and presentation is written using Overleaf, an open source cloud based Latex editor, while organizing for build up of notes and content of the report was managed using Google Drive and Google Docs.

Overall, the project was a huge learning experience that allowed multi-language thinking and was a confidence booster for future work in Control and Optimization technique (OT).

6 Supplemental Material

6.1 Github Link To Code

https://github.com/Sthitadhee/lost-target-search

6.2 Github Link To README.md

https://github.com/Sthitadhee/lost-target-search/blob/master/README.md

6.3 Link To Website

https://lost-target-search.herokuapp.com

List of Acronyms

ACO	ant colony optimization
BA	bayesian approach
CEO	cross entropy optimization
CSS	cascading style sheets
GB	global best
GD	gradient descent
HTML	hypertext markup language
JS	JavaScript
LB	local best
LTS	lost target search
MPSO	motion-encoded particle swarm optimization
NEXP	nondeterministic exponential
NP	nondeterministic polynomial
OT	optimization technique
PaaS	platform as a service
PDF	probability density function
PSO	particle swarm optimization
UAV	unmanned aerial vehicle
VS	visual studio

List of Figures

List of References

Bourgault, F., Furukawa, T., & Durrant-Whyte, H. F. (2003a). Coordinated decentralized search for a lost target in a bayesian world. *Proceedings 2003 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2003)(Cat. No. 03CH37453)*, *1*, 48–53.

Bourgault, F., Furukawa, T., & Durrant-Whyte, H. F. (2003b). Optimal search for a lost target in a bayesian world. *Field and service robotics*, 209–222.

Lanillos, P., Yañez-Zuluaga, J., Ruz, J. J., & Besada-Portas, E. (2013). A bayesian approach for constrained multi-agent minimum time search in uncertain dynamic domains. *Proceedings of the 15th annual conference on Genetic and evolutionary computation*, 391–398.

Phung, M. D., & Ha, Q. P. (2020). Motion-encoded particle swarm optimization for moving target search using uavs. *Applied Soft Computing*, *97*, 106705.

Wong, E.-M., Bourgault, F., & Furukawa, T. (2005). Multi-vehicle bayesian search for multiple lost targets. *Proceedings of the 2005 ieee international conference on robotics and automation*, 3169–3174.